Adding Asynchronous Events to Contexts

With all of that out of the way, we can add the ActionCable subscription back to the system, but we have a logistical problem to deal with first.

The logistical problem is that the ActionCable subscription needs to both be global and have external access to the existing VenueContext reducer. It needs to be global because multiple components may need access to the subscription data. But it also needs to be external from the VenueContext because the ActionCable subscription uses the concert_id, which is now stored as data in the state. But it will also need to use the dispatch function because the subscription receives data about what tickets have been held from the ActionCable channel, and it will need to use dispatch to update global state based on that data. This means the subscription needs to know about the reducer when it's defined, so it has access to dispatch, meaning it has to be external to the reducer.

We can manage this problem in a few different ways. The solution I chose for our purposes is to create a new context to make the subscription a separate part of global data, declared like this:

chapter_11/02/app/javascript/components/app.tsx

```
export const SubscriptionContext = React.createContext<Subscription>(null)

let appSubscription: Subscription = null

const initSubscription = (
  state: VenueState,
  dispatch: React.Dispatch<VenueAction>
): Subscription => {
  if (!appSubscription) {
    appSubscription = createConsumer().subscriptions.create(
      { channel: "ConcertChannel", concertId: state.concertId },
      {
        received(tickets) {
          dispatch({ type: "setTickets", tickets })
        },
      }
```